Image Captioning & Feature Extraction with BLIP-2

BLIP-2 (Bootstrapping Language-Image Pre-training 2) is a vision-language model from Salesforce Research that can generate natural language descriptions of images.

This notebook is part of an **ambiguity detection pipeline**: Image → BLIP-2 → MLP/Transformer. We use BLIP-2 to extract features that help detect **vague or ambiguous captions**.

We will:

1. Load the **Salesforce/blip2-opt-2.7b** model (~3.7 GB)
2. Generate a single caption from an image
3. Generate **multiple captions** with different sampling strategies to measure variance
4. Ask **VQA probing questions** to detect vagueness
5. Extract **embeddings** for the downstream MLP/Transformer classifier

Requirements: Google Colab with GPU runtime (T4 is sufficient).

```
!pip install transformers accelerate torch torchvision Pillow requests matplotlib -q
```

Import Libraries

```
import torch
from PIL import Image
import requests
from transformers import Blip2Processor, Blip2ForConditionalGeneration
import matplotlib.pyplot as plt

device = "cuda" if torch.cuda.is_available() else "cpu"
dtype = torch.float16 if device == "cuda" else torch.float32
print(f"Using device: {device} | dtype: {dtype}")
```

```
Using device: cuda | dtype: torch.float16
```

Load the Model

We use `Blip2Processor` for image preprocessing and tokenization, and `Blip2ForConditionalGeneration` for the model itself.

Note: The first run downloads ~3.7 GB of model weights. Subsequent runs use the Hugging Face cache.

```
model_name = "Salesforce/blip2-opt-2.7b"

processor = Blip2Processor.from_pretrained(model_name)
model = Blip2ForConditionalGeneration.from_pretrained(
    model_name,
    torch_dtype=dtype,
    # If you run out of GPU memory, uncomment the line below (requires: pip install bitsandbytes)
    # load_in_8bit=True,
)
model.to(device)
print(f"Model loaded: {model_name}")
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
processor_config.json: 100%  68.0/68.0 [00:00<00:00, 7.72kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
preprocessor_config.json: 100%  432/432 [00:00<00:00, 11.2kB/s]
The image processor of type `BlipImageProcessor` is now loaded as a fast processor by default, even if the model checkpoint was
config.json: 1.03k/? [00:00<00:00, 24.1kB/s]
tokenizer_config.json: 100%  882/882 [00:00<00:00, 36.2kB/s]
vocab.json: 798k/? [00:00<00:00, 10.7MB/s]
merges.txt: 456k/? [00:00<00:00, 11.5MB/s]
added_tokens.json: 100%  23.0/23.0 [00:00<00:00, 851B/s]
special_tokens_map.json: 100%  548/548 [00:00<00:00, 22.3kB/s]
tokenizer.json: 3.56M/? [00:00<00:00, 25.8MB/s]
model.safetensors.index.json: 122k/? [00:00<00:00, 3.14MB/s]
Download complete: 100%  15.0G/15.0G [02:43<00:00, 93.4MB/s]
Fetching 2 files: 100%  2/2 [02:43<00:00, 163.34s/it]
Loading weights: 100%  1247/1247 [00:59<00:00, 26.86it/s, Materializing param=vision_model.post_layernorm.weight]
generation_config.json: 100%  141/141 [00:00<00:00, 8.42kB/s]
Model loaded: Salesforce/blip2-opt-2.7b
```

Load an Image

By default we fetch a sample image from a URL. To use your own image, uncomment **Option B** and provide a local file path.

```
# Option A: Load from URL (default)
url = "https://storage.googleapis.com/sfr-vision-language-research/BLIP/demo.jpg"
image = Image.open(requests.get(url, stream=True).raw).convert("RGB")

# Option B: Load from a local file (uncomment to use)
# image = Image.open("path/to/your/image.jpg").convert("RGB")

plt.figure(figsize=(8, 6))
plt.imshow(image)
plt.axis("off")
plt.title("Input Image")
plt.show()
```

Input Image

Generate a Caption

Unconditional captioning — the model freely describes the image with no text prompt.

On CPU this may take 30–60 seconds.

```
inputs = processor(images=image, return_tensors="pt").to(device, dtype)

with torch.no_grad():
    generated_ids = model.generate(**inputs, max_new_tokens=50)

caption = processor.batch_decode(generated_ids, skip_special_tokens=True)[0].strip()
print(f"Generated caption: {caption}")
```

```
Generated caption: a woman sitting on the beach with a dog
```

Display Single Caption Result

```
fig, ax = plt.subplots(1, 1, figsize=(10, 7))
ax.imshow(image)
ax.axis("off")
ax.set_title(caption, fontsize=14, pad=15, wrap=True)
plt.tight_layout()
plt.show()
```

a woman sitting on the beach with a dog

Multi-Caption Generation

To detect **vague or ambiguous** images, we generate multiple captions using different decoding strategies:

Strategy	Parameters	Purpose
Greedy	`do_sample=False`	Baseline — most likely caption
Beam Search	`num_beams=5`	Best overall sequence
Nucleus Sampling	`top_p=0.9, temp=0.7`	Moderate randomness
High Temperature	`temp=1.2`	More creative/diverse
Nucleus + High Temp	`top_p=0.9, temp=1.2`	Maximum diversity

Key insight: If the captions vary wildly, the image may be ambiguous or the model is uncertain about its content. If they are all similar, the image has a clear, unambiguous interpretation.

```
inputs = processor(images=image, return_tensors="pt").to(device, dtype)

# Define decoding strategies
strategies = {
    "Greedy":              {"do_sample": False, "max_new_tokens": 50},
    "Beam Search (k=5)":   {"do_sample": False, "num_beams": 5, "max_new_tokens": 50},
    "Nucleus (p=0.9)":     {"do_sample": True, "top_p": 0.9, "temperature": 0.7, "max_new_tokens": 50},
    "High Temp (t=1.2)":   {"do_sample": True, "temperature": 1.2, "max_new_tokens": 50},
    "Nucleus + High Temp": {"do_sample": True, "top_p": 0.9, "temperature": 1.2, "max_new_tokens": 50},
}

captions = {}
with torch.no_grad():
    for name, params in strategies.items():
        generated_ids = model.generate(**inputs, **params)
        text = processor.batch_decode(generated_ids, skip_special_tokens=True)[0].strip()
        captions[name] = text
        print(f"[{name}]  {text}")

print(f"\nGenerated {len(captions)} captions.")
```

```
[Greedy]  a woman sitting on the beach with a dog
[Beam Search (k=5)]  a woman sitting on the beach with her dog
[Nucleus (p=0.9)]  woman on the beach with dog at sunset
[High Temp (t=1.2)]  man and dog petting in the water off the beach at sunrise and sunset
[Nucleus + High Temp]  a dog and girl playing together

Generated 5 captions.
```

```
from itertools import combinations

def word_overlap(caption_a, caption_b):
    """Compute Jaccard similarity (word overlap) between two captions."""
    words_a = set(caption_a.lower().split())
    words_b = set(caption_b.lower().split())
    if not words_a or not words_b:
        return 0.0
    intersection = words_a & words_b
    union = words_a | words_b
    return len(intersection) / len(union)

# Compute pairwise similarity across all generated captions
caption_texts = list(captions.values())
pairs = list(combinations(range(len(caption_texts)), 2))
similarities = [word_overlap(caption_texts[i], caption_texts[j]) for i, j in pairs]

avg_similarity = sum(similarities) / len(similarities) if similarities else 0.0
ambiguity_score = 1.0 - avg_similarity  # Higher = more ambiguous

print("Pairwise word-overlap similarities:")
for (i, j), sim in zip(pairs, similarities):
    print(f"  {list(captions.keys())[i]} vs {list(captions.keys())[j]}: {sim:.3f}")

print(f"\nAverage similarity: {avg_similarity:.3f}")
print(f"Ambiguity score:   {ambiguity_score:.3f}  (0 = identical captions, 1 = completely different)")
```

```
Pairwise word-overlap similarities:
  Greedy vs Beam Search (k=5): 0.889
  Greedy vs Nucleus (p=0.9): 0.600
  Greedy vs High Temp (t=1.2): 0.176
  Greedy vs Nucleus + High Temp: 0.167
  Beam Search (k=5) vs Nucleus (p=0.9): 0.545
  Beam Search (k=5) vs High Temp (t=1.2): 0.167
  Beam Search (k=5) vs Nucleus + High Temp: 0.154
  Nucleus (p=0.9) vs High Temp (t=1.2): 0.333
  Nucleus (p=0.9) vs Nucleus + High Temp: 0.077
  High Temp (t=1.2) vs Nucleus + High Temp: 0.125

Average similarity: 0.323
Ambiguity score:   0.677  (0 = identical captions, 1 = completely different)
```

VQA Probing Questions

We ask BLIP-2 targeted questions about the image. If answers are vague (e.g., "something", "not sure") or inconsistent, this signals ambiguity.

These Q&A pairs can also be passed as additional text features to the MLP/Transformer.

```
probing_questions = [
    "What is the main subject of this image?",
    "What is happening in this image?",
    "Where was this photo taken?",
    "How many people are in this image?",
    "What colors are prominent in this image?",
    "What is in the background?",
    "What emotion or mood does this image convey?",
]

vqa_answers = {}
with torch.no_grad():
    for question in probing_questions:
        prompt = f"Question: {question} Answer:"
        inputs = processor(images=image, text=prompt, return_tensors="pt").to(device, dtype)
        generated_ids = model.generate(**inputs, max_new_tokens=30)
        answer = processor.batch_decode(generated_ids, skip_special_tokens=True)[0].strip()
        vqa_answers[question] = answer
```

```
# Display as a formatted table
print(f"{'Question':<50} {'Answer'}")
print("-" * 80)
for q, a in vqa_answers.items():
    print(f"{q:<50} {a}")
```

```
Question                                           Answer
--------------------------------------------------------------------------------
What is the main subject of this image?            Question: What is the main subject of this image? Answer: A woman and her dog
What is happening in this image?                   Question: What is happening in this image? Answer: A woman is petting her dog
Where was this photo taken?                        Question: Where was this photo taken? Answer: The beach in the background is
How many people are in this image?                 Question: How many people are in this image? Answer: 1
What colors are prominent in this image?           Question: What colors are prominent in this image? Answer: Red, blue, green,
What is in the background?                         Question: What is in the background? Answer: A beautiful woman and her dog on
What emotion or mood does this image convey?       Question: What emotion or mood does this image convey? Answer: happiness
```

Embedding Extraction

Instead of passing raw caption text to the MLP/Transformer, we can extract **BLIP-2's internal representations** — dense vectors that encode both visual and linguistic information.

These embeddings are far richer features for a downstream classifier than text strings alone. We extract:

- **Vision embeddings** from BLIP-2's Q-Former (the bridge between the image encoder and the language model)
- **Language model hidden states** from the last layer of the OPT decoder

The embeddings are saved as `.pt` files so your MLP team can load them with `torch.load()`.

```
pixel_values = processor(images=image, return_tensors="pt")["pixel_values"].to(device, dtype)

with torch.no_grad():
    # Extract Q-Former vision embeddings (the bridge between vision encoder and LLM)
    # The vision encoder and qformer can be called directly or via get_qformer_features if available
    try:
        vision_outputs = model.get_qformer_features(pixel_values=pixel_values)
        qformer_embeds = vision_outputs[0]
    except AttributeError:
        # Fallback for some transformer versions
        vision_outputs = model.vision_model(pixel_values=pixel_values)
        image_embeds = vision_outputs[0]
        image_atts = torch.ones(image_embeds.size()[:-1], dtype=torch.long).to(device)
        query_tokens = model.query_tokens.expand(image_embeds.shape[0], -1, -1)
        qformer_outputs = model.qformer(
            query_embeds=query_tokens,
            encoder_hidden_states=image_embeds,
            encoder_attention_mask=image_atts,
            return_dict=True,
        )
        qformer_embeds = qformer_outputs.last_hidden_state  # (batch, num_query_tokens, hidden_dim)

    # Extract language model hidden states by running a forward pass
    inputs = processor(images=image, return_tensors="pt").to(device, dtype)
    outputs = model.generate(
        **inputs,
        max_new_tokens=50,
        output_hidden_states=True,
        return_dict_in_generate=True,
    )
    # Get the last hidden state from the final generation step
    last_hidden = outputs.hidden_states[-1][-1]  # last step, last layer

print(f"Q-Former embeddings shape: {qformer_embeds.shape}")
print(f"  -> {qformer_embeds.shape[1]} query tokens x {qformer_embeds.shape[2]} dims")
print(f"\nLLM last hidden state shape: {last_hidden.shape}")

# Save embeddings for the MLP team
torch.save({
    "qformer_embeddings": qformer_embeds.cpu(),
    "llm_last_hidden_state": last_hidden.cpu(),
}, "sample_embeddings.pt")
print("\nEmbeddings saved to 'sample_embeddings.pt'")
```

```
Q-Former embeddings shape: torch.Size([1, 32, 768])
  -> 32 query tokens x 768 dims

LLM last hidden state shape: torch.Size([1, 1, 2560])

Embeddings saved to 'sample_embeddings.pt'
```

Putting It All Together

The `extract_features()` function wraps everything into a single call. For any image, it returns:

- `caption` — the greedy baseline caption
- `multi_captions` — dict of captions from different decoding strategies
- `ambiguity_score` — 0 (clear) to 1 (ambiguous) based on caption variance
- `vqa_answers` — dict of probing question → answer pairs
- `qformer_embeddings` — Q-Former vision embeddings tensor
- `llm_hidden_state` — language model last hidden state tensor

This dict is the complete feature set your MLP/Transformer team can consume.

```
def extract_features(img, model=model, processor=processor, device=device, dtype=dtype):
    """
    Extract all BLIP-2 features from an image for the ambiguity detection pipeline.

    Args:
        img: PIL Image (RGB)

    Returns:
        dict with keys: caption, multi_captions, ambiguity_score,
                        vqa_answers, qformer_embeddings, llm_hidden_state
    """
    # --- 1. Multi-caption generation ---
    img_inputs = processor(images=img, return_tensors="pt").to(device, dtype)

    strategies = {
        "Greedy":              {"do_sample": False, "max_new_tokens": 50},
        "Beam Search (k=5)":   {"do_sample": False, "num_beams": 5, "max_new_tokens": 50},
        "Nucleus (p=0.9)":     {"do_sample": True, "top_p": 0.9, "temperature": 0.7, "max_new_tokens": 50},
        "High Temp (t=1.2)":   {"do_sample": True, "temperature": 1.2, "max_new_tokens": 50},
        "Nucleus + High Temp": {"do_sample": True, "top_p": 0.9, "temperature": 1.2, "max_new_tokens": 50},
    }

    multi_captions = {}
    with torch.no_grad():
        for name, params in strategies.items():
            gen_ids = model.generate(**img_inputs, **params)
            text = processor.batch_decode(gen_ids, skip_special_tokens=True)[0].strip()
            multi_captions[name] = text

    # Baseline caption (greedy)
    baseline_caption = multi_captions["Greedy"]

    # --- 2. Ambiguity score ---
    caption_texts = list(multi_captions.values())
    pairs = list(combinations(range(len(caption_texts)), 2))
    sims = [word_overlap(caption_texts[i], caption_texts[j]) for i, j in pairs]
    amb_score = 1.0 - (sum(sims) / len(sims)) if sims else 0.0

    # --- 3. VQA probing ---
    questions = [
        "What is the main subject of this image?",
        "What is happening in this image?",
        "Where was this photo taken?",
        "How many people are in this image?",
        "What colors are prominent in this image?",
        "What is in the background?",
        "What emotion or mood does this image convey?",
    ]
    answers = {}
    with torch.no_grad():
        for q in questions:
            prompt = f"Question: {q} Answer:"
            q_inputs = processor(images=img, text=prompt, return_tensors="pt").to(device, dtype)
```

```
            gen_ids = model.generate(**q_inputs, max_new_tokens=30)
            answers[q] = processor.batch_decode(gen_ids, skip_special_tokens=True)[0].strip()

    # --- 4. Embeddings ---
    pixel_vals = processor(images=img, return_tensors="pt")["pixel_values"].to(device, dtype)
    with torch.no_grad():
        try:
            qf_out = model.get_qformer_features(pixel_values=pixel_vals)
            qf_embeds = qf_out[0].cpu()
        except AttributeError:
            vision_outputs = model.vision_model(pixel_values=pixel_vals)
            image_embeds = vision_outputs[0]
            image_atts = torch.ones(image_embeds.size()[:-1], dtype=torch.long).to(device)
            query_tokens = model.query_tokens.expand(image_embeds.shape[0], -1, -1)
            qformer_outputs = model.qformer(
                query_embeds=query_tokens,
                encoder_hidden_states=image_embeds,
                encoder_attention_mask=image_atts,
                return_dict=True,
            )
            qf_embeds = qformer_outputs.last_hidden_state.cpu()

        gen_out = model.generate(
            **img_inputs, max_new_tokens=50,
            output_hidden_states=True, return_dict_in_generate=True,
        )
        lm_hidden = gen_out.hidden_states[-1][-1].cpu()

    return {
        "caption": baseline_caption,
        "multi_captions": multi_captions,
        "ambiguity_score": amb_score,
        "vqa_answers": answers,
        "qformer_embeddings": qf_embeds,
        "llm_hidden_state": lm_hidden,
    }


# --- Demo on the sample image ---
features = extract_features(image)

print(f"Caption:          {features['caption']}")
print(f"Ambiguity Score:  {features['ambiguity_score']:.3f}")
```